LAUNCH CONTROL SYSTEM

ABSTRACT

This project presents a microcontroller-based Launch Control System that emulates the functional behaviour of a real missile safety-arming and command-control unit using an Arduino Uno, I²C 16×2 LCD, relay module, dual buzzers and a bank of status LEDs. The system powers up in a secure standby state and can be activated only after successful entry of a fixed four-digit password, which is entered using a single toggle switch through multi-press detection. Once authenticated, the controller performs an automatic health check of the relay and audio indicators and then executes a staged launch sequence. Key stages shown on the LCD include system boot, communication link establishment, navigation alignment (representing initialization), arming preparation, safety verification, countdown with live abort facility, launch ignition, mid-course flight, target search, target lock, trajectory tracking with distance indication, and final target impact and mission success. Three green LEDs provide positive status feedback (standby, arming and launch, mission success), while three red LEDs indicate warning and fault conditions (password error, safe mode, countdown alert and loop-mode stop window), and the relay produces realistic arming and ignition clicks. Mission statistics are stored in EEPROM and a post-mission menu allows the operator to choose between single-run restart and automatic loop mode. Overall, the prototype demonstrates how concepts from electronics and instrumentation engineering—digital control, signal conditioning, interlock logic, human-machine interface design and non-volatile data logging—can be integrated to build a safe, sequenced launch control and safety-arming mechanism.

Table of Contents

CHAPTER-1

1.1 INTRODUCTION

A Launch Control System (LCS) is a critical part of any aerospace, missile, or defense technology where a launch is permitted only after fulfilling strict authentication, safety arming, sequencing, and command verification. In real missile systems, a launch command cannot be initiated randomly; it requires secure password authentication, safety arming circuits, communication system checks, navigation alignment, and confirmation from the command headquarters. Only when all systems are verified and conditions are safe, the missile enters the arming and ignition stage and proceeds toward launch.

In this project, we simulate a Missile Launch Control System using Arduino, where all these essential mechanisms are modeled using simple electronic components like microcontroller, relay, buzzers, LCD, LEDs, and toggle switch. The system automatically displays different stages such as System Standby, Password Authentication, Health Check, Arming Preparation, Countdown, Launch Active, Target Locking, Tracking, Impact, and Mission Success, making it a close representation of real-world missile launch operations.

The Arduino Uno microcontroller functions as the Central Command Unit, processing all tasks and controlling the Human-Machine Interface (LCD display), buzzers (audible alerts), relay (ignition simulation), and 3 LED indicators (stage monitoring). The toggle switch works as a single multi-functional control button used for password input, abort, restart, and loop-mode selection based on press count. This simulates a secure command-based launch mechanism often used in military and industrial safety systems.

This project is highly relevant to Electronics and Instrumentation Engineering because it demonstrates practical applications of control systems, safety interlocking, signal conditioning, relay interfacing, PWM control, EEPROM-based memory, real-time user interface design, automation, and communication logic. It combines theory with hands-on implementation and gives a complete understanding of how a real missile launch sequencing and tracking system works.

1.2 CHALLENGES

1. Secure Authentication with Minimal Hardware

Real missile systems use encrypted command links, biometric locks, and multiple-level approvals.

Simulating this using only one toggle switch as input device was challenging because it had to capture multiple inputs, detect press patterns, serve as abort, confirm, and selection control — all without any keypad.

2. Realistic Simulation of Arming and Ignition

A missile cannot launch immediately upon command — it undergoes Arming Preparation, Safety Checks, and Ignition Priming.Simulating this with only relay pulses, LEDs, buzzer sound patterns, and time delays while keeping it realistic was hard.

3. Sequence Synchronization and Abort Capability

The system needed to simulate live countdown, abort handling, target locking, flight animation, and distance tracking all on a 16x2 LCD display, which has limited space.Making it event-based instead of a normal timed delay process was complicated.

4. Memory Storage & Loop Mode Execution

The system had to remember mission count using EEPROM, repeat execution in Loop Mode, and still allow restart or exit using just a toggle switch.Managing this control logic with minimal input hardware was a major challenge.

1.3 PROJECT AIM

The aim of this project is to design, simulate, and demonstrate a microcontroller-based missile launch control system that accurately models secure access, safety interlocking, command verification, ignition simulation, target locking, and mission success sequence, using simple electronic components.

✓ To implement password-protected access using a toggle switch as the only input device
✓ To simulate arming preparation using relay activation and blinking LED indicators
✓ To perform a countdown launch with live abort capability
✓ To demonstrate target detection, trajectory tracking, and impact simulation
✓ To display launch status using LCD-based Human Machine Interface (HMI)
✓ To use EEPROM to store number of successful missions
✓ To include Loop Mode & Restart Mode, simulating continuous mission readiness
✓ To replicate real Safety Arming and Command-Control Mechanism from missile systems

CHAPTER 2

2.1 PROCEDURE

Step 1 — Power ON (Standby Mode)

- System turns ON when connected to USB power
- LCD displays: SYSTEM STANDBY
- LED_G1 glows dim indicating readiness
- Waits for Start Command (toggle switch press)

Step 2 — Authentication (Password Entry)

- Operator enters password using toggle switch
- 1 Press = 1, 2 Press = 2, 3 Press = 3, 4 Press = 4
- LCD continuously displays password input
- System allows max 4 digits only
- If correct → ACCESS GRANTED
- If wrong → SAFE MODE (entry denied)

Step 3 — System Health Check

- Relay, LEDs, and buzzers are tested
- LCD displays "SYSTEM CHECKING…"
- Ensures buzzer and relay functional
- If any component fails → ERROR INDICATION

Step 4 — Initial System Boot

- LCD: "SYSTEM BOOTING…"
- Establishes simulation of control module
- Arduino initializes mission data from EEPROM

Step 5 — Communication Link Verification

- LCD: "LINK TO GCS (Ground Control Station) OK"
- Radar beep indicates communication established

Step 6 — Navigation Alignment (Simulated INS Setup)

- LCD: "NAV ALIGNMENT IN PROGRESS"
- LED_G1 slowly fades in/out (PWM)
- Simulates real missile's Inertial Navigation System (INS) calibration

Step 7 — Arming Preparation (Relay Control)

- Relay turns ON & OFF repeatedly (stretching)
- LEDs pulse indicating arming process
- Relay simulates ignition circuit priming
- Display: "ARM PREP IN PROGRESS"

Step 8 — Safety Verification

- LCD: "SAFETY SYSTEM VERIFIED"
- Launch enabled only if safe and armed

Step 9 — Countdown with Abort Option

◈ Countdown displayed from 10 to 1
◈ LED_G2 blinks faster as time shortens
◈ Buzzer beeps once every second
◈ If button pressed → ABORTED – SAFE MODE
◈ If no cancel input → auto launch

Step 10 — Launch & Ignition Simulation

◈ LCD: "LAUNCH ACTIVE"
◈ Relay turns ON (stretching) for ignition
◈ Continuous buzzer alert
◈ LED_G2 turns fully ON (Launch Indicator)

Step 11 — In-Flight Simulation

◈ LCD displays:
- MID-AIR FLIGHT
- TARGET SEARCH
- TARGET LOCKED

TRAJECTORY TRACKING
◈ Distance reduces: 8km → 4km → 2km → 0km
◈ Radar beep for tracking
◈ LED flashes

Step 12 — Impact & Mission Success

◈ LCD: "TARGET AMBUSHED"
◈ Buzzer celebration beep
◈ LED_G3 pulses (SUCCESS)
◈ EEPROM stores mission count
◈ Menu appears: Restart Mode or Loop Mode

Launch Cycle Timing Overview

Stage Time Duration:-	
Standby & Password	10–20 sec
Health & Boot	5 sec
Arming Preparation	7 sec
Countdown	10 sec
Launch & Flight	8–10 sec
Mission Success & Replay	5–10 sec

2.2 Design specifications: -

This project is designed to function as a microcontroller-based Launch Control System, simulating real missile launch operations. The system follows a sequential, event-driven, safety-controlled process that mimics real-world Command and Control (C2) and Safety Arming systems.

Features and their specifications: -

Control Unit: - Arduino Uno (ATmega328P)
Display Type: - 16x2 LCD with I2C
Input Method: - Single Toggle Switch (Multi-press Detection)
Output Devices: - 3 Green LEDs, 2 Buzzers, Relay
Power Source: - USB Power / 5V DC
Authentication: - 4-digit password using multi-press logic
Relay Function: - Simulate arming and ignition signal
Buzzer Alerts: - Alarm, Countdown, Launch Sound, Target Impact
LED Indications: - Standby, Launch Active, Mission Success
Abort Feature: - Press switch during countdown to cancel launch
EEPROM Usage: - Stores total number of successful missions
Launch Modes: - Restart Mode & Loop Mode

CHAPTER 3

3.1 List of components used in Launch Control System: -

S.No	Component Name	Quantity	Purpose
1	Arduino Uno Microcontroller	1	Main control and processing unit
2	16x2 I2C LCD Display	1	Visual display for launch stages and status
3	5V Relay Module (1-channel)	1	Simulated ignition/arming mechanism
4	Toggle Switch / Push Button	1	Password, control, abort, restart input
5	Active Buzzers	2	Countdown, launch, alert, mission success
6	Green LEDs	3	Visual stage indicators
7	Resistors (220Ω)	3	Current limiting for LEDs and buzzers
8	Breadboard (400 points)	1	Temporary prototyping circuit
9	Jumper Wires	20	Wiring and connections
10	USB Cable Type B	1	Power and programming via laptop
11	Arduino IDE Software	1	Code compilation and upload

ARDUINO UNO: -

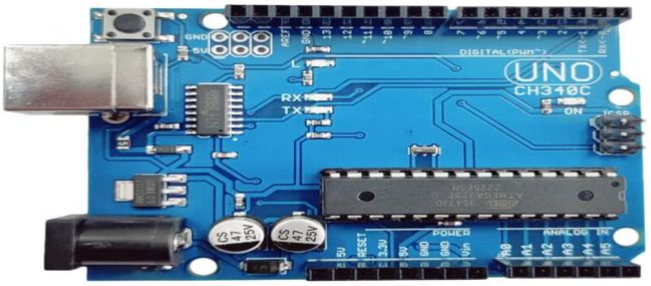
1. Arduino Uno Microcontroller: - Arduino Uno is the central brain of the entire project. It is based on the ATmega328P microcontroller and is responsible for controlling all devices including LCD, relay, LEDs, buzzers, and input switch.

Features:

- 14 Digital I/O pins (6 PWM pins)
- Analog input pins
- 16 MHz clock speed
- Operates on 5V power
- Built-in EEPROM for storing mission count
- USB programmable using Arduino IDE

Role in Project:

- Controls password authentication
- Activates Relay for arming and launch
- Controls PWM effects for LEDs & buzzers
- Displays messages on LCD (HMI)
- Runs countdown, target tracking, and resets



2. 16x2 I2C LCD Display: - This is a Human Machine Interface (HMI) that shows real-time status of launch operations.

Shows following stages:

- SYSTEM STANDBY

- ENTER PASSWORD
- ACCESS GRANTED / DENIED
- ARM PREP IN PROGRESS
- COUNTDOWN RUNNING
- LAUNCH ACTIVE
- TARGET LOCKED
- IMPACT & MISSION SUCCESS

3. Relay Module (5V, 1 Channel): - A relay is an electromagnetic switch used in this project to simulate arming and ignition.

Usage in launch system:

- Relay pulses ON/OFF during arming
- Stays ON for ignition during launch active
- Simulates electrical firing mechanism

4. Toggle Switch (Push Button): - Single switch used for multiple functions using press-count logic.

1 to 4 presses: -Enter Password digit

During countdown: -ABORT LAUNCH
2 presses: - Confirm selection (Mode Menu)
3 presses: - Change mode (Restart / Loop)

5. Active Buzzers (x2): -Used for alarm and status indication using PWM for realistic tones.

Single beep	Key press / confirmation
Double beep	Verification successful
Slow beep	Countdown in progress
Fast beep	Final countdown warning
Continuous beep	Launch / ignition
Pulse beep	Target locked / mission success

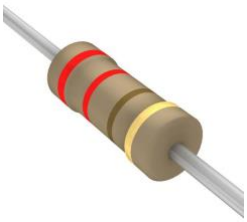
6. Green LEDs (x3): -Used as visual indication of launch stages.

LED Pin		Indication Purpose
LED_G1	D3	Standby / Nav Alignment
LED_G2	D6	Arming / Countdown / Launch Active
LED_G3	D9	Mission Success / Loop Mode

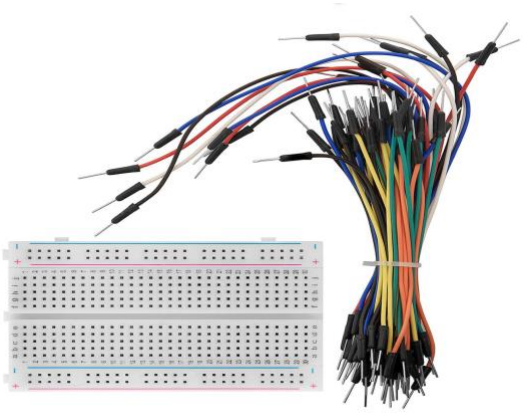
7. Resistors (220Ω): -Used to limit current through LEDs and buzzers preventing damage.

Without resistors → LED burns out / low lifespan

Ensures correct voltage drop — safe working conditions.



8. Jumper Wires and Breadboard: -Used to make connections without soldering. Helps in exploration, modification, and safety testing.



9. USB Cable & Arduino IDE

Used for uploading code and providing power supply.

3.3 working of circuit: -

The Launch Control System works based on the principles of sequential automation, safety interlock, human-machine interface, and command-based ignition simulation. The system uses the Arduino Uno microcontroller as the central control unit, which coordinates all operations such as password authentication, relay switching, LED indication, countdown timer, buzzer alerts, LCD display, and mission status updates.Once the system is powered ON using USB, it automatically enters Standby Mode, where the LCD shows "SYSTEM STANDBY" and the mission count (stored in EEPROM). LED_G1 glows dimly, indicating that the system is powered and safe. The system waits for the operator to press the toggle switch (single button input) to begin authentication.

Password Authentication Stage: -Instead of a keypad, the user enters a 4-digit password (1234) using the toggle switch:

1 press → Digit 1; 2 presses → Digit 2; 3 presses → Digit 3; 4 presses → Digit 4

Each digit entry is displayed live on the LCD. If the password matches, the system displays ACCESS GRANTED and proceeds. Otherwise, it displays ACCESS DENIED, activates buzzer alerts, and stays in Safe Mode, similar to real military launch authentication systems.

System Health Check & Boot Sequence: -Before launch, the system performs an automatic hardware health check, testing:

- Buzzers (sound output)
- Relay (ignition switching)
- LEDs (stage indication)
- LCD display response

This simulates real missile systems where communication devices, ignition circuits, and tracking modules must be verified before launch.

Communication & Navigation Alignment: - Next, the system simulates: Link Establishment with Ground Control Station (GCS)

Navigation Alignment (GPS initialization): - These processes are displayed on LCD with radar beeps and glowing LED animations. In real systems, this step ensures missile alignment, trajectory calculation, and navigation before armed readiness.

Arming Preparation (Relay-based safety simulation): -In real missiles, arming circuits are activated using electro-explosive devices (EED) and squib ignition using relay-controlled charge.In this project, the relay simulates that arming process by turning ON/OFF repeatedly for 7 seconds. LEDs pulsate along with relay to indicate active arming.

This stage represents:

✓ Ignition circuit preparation ✓ Safety interlock verification ✓ Weapon activation readiness

Countdown with Abort Option: -The Arduino begins a 10-second countdown. Buzzer beeps every second LED_G2 blinks fast. LCD displays time remaining. Operator can press toggle switch to ABORT launch. The ability to cancel the launch during countdown is an important real-world safety feature used in missile safety-arming mechanisms. Launch and Ignition - If countdown is completed without

interruption: ✓ Relay turns ON continuously → Ignition simulation ✓ Buzzer gives continuous launch alert ✓ LCD displays LAUNCH ACTIVE ✓ LED_G2 glows fully

This replicates the ignition command triggering in missile launch systems. In-flight Simulation and Target Tracking. After launch, the system simulates: Mid-Air Flight, Target Search, Target Lock, Trajectory Tracking. LCD displays moving missile animation, distance reduction (8 km → 0 km), radar beeps, and signal strength — simulating live tracking using INS/GPS.

Impact and Mission Success: - When distance reaches 0 km, the system displays: ✓ TARGET AMBUSHED ✓ MISSION SUCCESS ✓ Updates total mission count in EEPROM ✓ LED_G3 pulses gently (success visual effect) Post-Mission Options After mission completion, system displays menu:- RESTART Mode: Run only once and stop. LOOP Mode: Continuous missions until toggle switch is pressed. This simulates multiple launch cycles, training simulation, or repetitive testing mode. Key Concepts Demonstrated in Project. Human Machine Interface - LCD Display; Safety Interlock - Password & Abort System; Relay Switching - Ignition Simulation; PWM Control - LED and Buzzer Pulse; EEPROM Data Storage - Mission Count; Sequential Automation - Launch Sequence; Signal Conditioning - Active Relay & Buzzer; Toggle Switch Multi-Press Control- User Input

CHAPTER 4

4.1 source code: -

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <EEPROM.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
// ------- Pins -------
#define BUZZER1 A3
#define BUZZER2 8
#define RELAY   5        // Relay IN on D5
#define START_BUTTON 2    // Toggle switch, active LOW
// Green LEDs only
#define LED_G1 3   // Standby / Safe / Error / Nav Align
#define LED_G2 6   // Countdown / Arm Prep / Launch active
#define LED_G3 9   // Mission success / Loop mode indication
// ------------ Global states ------------
const String correctPassword = "1234";
String enteredPassword = "";
bool authenticated = false;
int missionCount = 0;
bool stopLoop = false;
bool aborted = false;
int countdownTime = 10;
int dist0 = 8, dist1 = 4, dist2 = 2, dist3 = 0;
// Custom Icons for LCD
byte missileChar[8]  = {B00100,B01110,B11111,B00100,B01110,B11111,B10101,B00100};
byte radarChar[8]    = {B00000,B00100,B01110,B10101,B00100,B00100,B00000,B00000};
byte targetChar[8]   = {B00000,B00100,B01110,B11111,B01110,B00100,B00000,B00000};
byte explosionChar[8] = {B00000,B10101,B01110,B11111,B01110,B10101,B00000,B00000};
```

```
byte successChar[8]   = {B00000,B00001,B00010,B10100,B01000,B00000,B00000,B00000};
byte blockChar[8]     = {B11111,B11111,B11111,B11111,B11111,B11111,B11111,B11111};
// ---------- BUZZER HELPERS ----------
void beepBoth(int freq, int durationMs) {
  long period = 1000000L / freq;
  long cycles = (long)durationMs * 1000L / period;
  for (long i = 0; i < cycles; i++) {
    digitalWrite(BUZZER1, HIGH);
    digitalWrite(BUZZER2, HIGH);
    delayMicroseconds(period / 2);
    digitalWrite(BUZZER1, LOW);
    digitalWrite(BUZZER2, LOW);
    delayMicroseconds(period / 2);
  }
}
void shortBeep()            { beepBoth(2000, 150); }
void doubleBeep()            { shortBeep(); delay(120); shortBeep(); }
void radarBeep()            { beepBoth(1600, 150); }
void continuousBeep(int t)   { beepBoth(2500, t); }
void buzzersOff()           { digitalWrite(BUZZER1, LOW); digitalWrite(BUZZER2, LOW); }
// ---------- LED HELPERS ----------
void allLedsOff() {
  analogWrite(LED_G1, 0);
  analogWrite(LED_G2, 0);
  analogWrite(LED_G3, 0);
}
// “Safe mode / error” flashing using LED_G1
void safeModeFlashGreen() {
  for (int i = 0; i < 6; i++) {
    analogWrite(LED_G1, 255);
    delay(150);
    analogWrite(LED_G1, 0);
    delay(150);
  }
}
// Success effect using LED_G3
void successPulseGreen() {
  for (int k = 0; k < 3; k++) {
    for (int b = 0; b <= 255; b += 15) {
      analogWrite(LED_G3, b);
      delay(15);
    }
    for (int b = 255; b >= 0; b -= 15) {
      analogWrite(LED_G3, b);
      delay(15);
    }
  }
  analogWrite(LED_G3, 0);
}

// ---------- PROGRESS BAR ----------
void progressBar(int msTotal) {
  lcd.setCursor(0, 1);
  for (int i = 0; i < 16; i++) {
    lcd.write(byte(5));
    delay(msTotal / 16);
  }
```

```
}
// ---------- EEPROM HELPERS ----------
void loadMissionCount() {
  EEPROM.get(10, missionCount);
  if (missionCount < 0 || missionCount > 10000) missionCount = 0;
}
void saveMissionCount() {
  EEPROM.put(10, missionCount);
}
// ---------- MULTI-PRESS READER ----------
int getPressCount(unsigned long windowMs) {
  int count = 0;
  bool prev = digitalRead(START_BUTTON);
  unsigned long start = millis();

  while (millis() - start < windowMs) {
    bool state = digitalRead(START_BUTTON);
    if (prev == HIGH && state == LOW) {
      count++;
      delay(150);
    }
    prev = state;
  }
  return count;
}
// ---------- SINGLE DIGIT ENTRY ----------
int getSingleDigit(unsigned long waitTime = 2000) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("ENTER DIGIT...");
  lcd.setCursor(0, 1);
  lcd.print("Press 1-4 times");
  int count = 0;
  bool prev = HIGH;
  unsigned long start = millis();
  while (millis() - start < waitTime) {
    bool state = digitalRead(START_BUTTON);
    if (prev == HIGH && state == LOW) {
      count++;
      delay(200);
    }
    prev = state;
  }

  if (count >= 1 && count <= 4) return count;
  return -1;
}
// ---------- PASSWORD ENTRY ----------
void enterPassword() {
  enteredPassword = "";
  authenticated = false;
  unsigned long lastActivity = millis();
  allLedsOff();
  analogWrite(LED_G1, 50);   // faint green while entering
  while ((int)enteredPassword.length() < 4) {
```

```
    int digit = getSingleDigit();
    if (digit != -1) {
      enteredPassword += String(digit);
      lastActivity = millis();
      shortBeep();
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("ENTER PASS:");
      lcd.setCursor(0, 1);
      lcd.print(enteredPassword);
      for (int i = enteredPassword.length(); i < 4; i++) lcd.print("_");
      delay(800);
    }
    if (millis() - lastActivity > 10000) {
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("TIMEOUT!");
      safeModeFlashGreen();
      delay(1500);
      authenticated = false;
      return;
    }
  }
  if (enteredPassword == correctPassword) {
    authenticated = true;
    lcd.clear();
    lcd.print("ACCESS GRANTED");
    allLedsOff();
    analogWrite(LED_G3, 200);
    doubleBeep();
    delay(1500);
    analogWrite(LED_G3, 0);
  } else {
    authenticated = false;
    lcd.clear();
    lcd.print("ACCESS DENIED");
    safeModeFlashGreen();
    continuousBeep(1500);
    delay(1500);
  }
}
// ---------- STANDBY ----------
void standbyMode() {
  allLedsOff();
  analogWrite(LED_G1, 80);  // standby green dim
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SYSTEM STANDBY");
  lcd.setCursor(0, 1);
  lcd.print("RUNS: ");
  lcd.print(missionCount);
  while (digitalRead(START_BUTTON) == HIGH) {
    // wait for press
  }
  delay(300);
  shortBeep();
}
```

```
// ---------- HEALTH CHECK ----------
void healthCheck() {
  allLedsOff();
  analogWrite(LED_G1, 120);
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("HEALTH CHECK");
  lcd.setCursor(0,1);
  lcd.print("TESTING BUZZER");
  shortBeep();
  delay(1000);
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("HEALTH CHECK");
  lcd.setCursor(0,1);
  lcd.print("TESTING RELAY");
  digitalWrite(RELAY, HIGH);
  delay(400);
  digitalWrite(RELAY, LOW);
  shortBeep();
  delay(1000);
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("ALL SYSTEMS");
  lcd.setCursor(0,1);
  lcd.print("OK");
  lcd.write(byte(4));
  doubleBeep();
  delay(1500);
  allLedsOff();
}
// ---------- BOOT & PREP STAGES ----------
void stageInitialBoot() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("SYSTEM BOOTING");
  lcd.setCursor(0,1);
  lcd.print("SELF CHECK...");
  shortBeep();
  progressBar(2000);
}
void stageCommsLink() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("LINK TO GCS ");
  lcd.write(byte(1));
  lcd.setCursor(0,1);
  lcd.print("COMMS STABLE");
  radarBeep();
  delay(2000);
}
void stageSystemReady() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("LAUNCH CONTROL");
```

```
  lcd.setCursor(0,1);
  lcd.print("SYSTEM READY ");
  lcd.write(byte(0));
  doubleBeep();
  delay(2000);
}
void stageNavAlign() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("NAV ALIGNMENT");
  lcd.setCursor(0,1);
  lcd.print("IN PROGRESS...");
  progressBar(3000);
  radarBeep();
  delay(2000);
}
// ---------- ARM PREP (7 seconds, green + relay) ----------
void stageArmPrep() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("ARM PREP IN");
  lcd.setCursor(0, 1);
  lcd.print("PROGRESS ");
  lcd.write(byte(0));

  for (int i = 0; i < 7; i++) {
    analogWrite(LED_G1, 255);
    analogWrite(LED_G2, 255);
    analogWrite(LED_G3, 255);
    digitalWrite(RELAY, HIGH);
    shortBeep();
    delay(500);
    analogWrite(LED_G1, 0);
    analogWrite(LED_G2, 0);
    analogWrite(LED_G3, 0);
    digitalWrite(RELAY, LOW);
    delay(500);
  }
  digitalWrite(RELAY, LOW);
  allLedsOff();
}
void stageSafetyCheck() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("SAFETY SYSTEM");
  lcd.setCursor(0,1);
  lcd.print("VERIFIED ");
  lcd.write(byte(4));
  doubleBeep();
  delay(2000);
}
// ---------- COUNTDOWN ----------
bool stageCountdown() {
  aborted = false;
  for (int t = countdownTime; t >= 1; t--) {
    lcd.clear();
    lcd.setCursor(0,0);
```

```
    lcd.print("COUNTDOWN: ");
    lcd.print(t);
    lcd.setCursor(0,1);
    lcd.print("PRESS TO ABORT");
    // Green LED2 as warning
    if (t > 5) analogWrite(LED_G2, 180);
    else       analogWrite(LED_G2, 255);
    unsigned long start = millis();
    while (millis() - start < 900) {
      if (digitalRead(START_BUTTON) == LOW) {
        aborted = true;
        break;
      }
    }
    analogWrite(LED_G2, 0);
    if (aborted) {
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("LAUNCH ABORTED");
      lcd.setCursor(0,1);
      lcd.print("SAFE MODE");
      continuousBeep(1500);
      safeModeFlashGreen();
      delay(1500);
      return false;
    }
    if (t > 5) shortBeep();
    else doubleBeep();
  }
  return true;
}
// ---------- LAUNCH & FLIGHT ----------
void stageLaunch() {
  lcd.clear();
  lcd.setCursor(1,0);
  lcd.print("LAUNCH ACTIVE");
  lcd.setCursor(0,1);
  lcd.print("IGNITION ");
  lcd.write(byte(0));
  analogWrite(LED_G2, 255);
  digitalWrite(RELAY, HIGH);
  continuousBeep(4000);
  digitalWrite(RELAY, LOW);
  analogWrite(LED_G2, 0);
}
void stageMidAirFlight() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("MID-COURSE");
  lcd.setCursor(0,1);
  lcd.print("FLIGHT STABLE");
  for (int i = 0; i < 5; i++) {
    radarBeep();
    delay(600);
  }
```

```
}
void stageTargetSearch() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("TARGET SEARCH ");
  lcd.write(byte(1));
  lcd.setCursor(0,1);
  lcd.print("SCANNING AREA");
  for (int i = 0; i < 5; i++) {
    radarBeep();
    delay(500);
  }
}
void stageTargetLocking() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("TARGET LOCKING");
  lcd.setCursor(0,1);
  lcd.print("ACQ SIGNAL ");
  lcd.write(byte(1));
  for (int i = 0; i < 4; i++) {
    radarBeep();
    delay(400);
  }
}
void stageTargetLocked() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("TARGET LOCKED ");
  lcd.write(byte(2));
  lcd.setCursor(0,1);
  lcd.print("READY TO ATTACK");
  for (int i = 0; i < 3; i++) {
    doubleBeep();
    delay(300);
  }
}
// ---------- TRAJECTORY ----------
void stageMissileTrajectoryAnimation() {
  String frames[4] = {
    "R----->-------->",
    "--->R--------->",
    "--->--->R----->",
    "--------->--->R"
  };
  int distances[4]  = {dist0, dist1, dist2, dist3};
  int signalBars[4] = {4, 6, 8, 10};
  for (int i = 0; i < 4; i++) {
    lcd.clear();
    lcd.setCursor(0,0);
    if (distances[i] > 0) lcd.print("TRACKING...");
    else                  lcd.print("APPROACHING TGT");
    lcd.setCursor(0,1);
    for (int c = 0; c < 16 && c < (int)frames[i].length(); c++) {
      if (frames[i][c] == 'R') lcd.write(byte(0));
      else lcd.print(frames[i][c]);
    }
```

```
    delay(800);
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("DIST: ");
    lcd.print(distances[i]);
    lcd.print("km");
    lcd.setCursor(0,1);
    lcd.print("SIG: ");
    for (int s = 0; s < signalBars[i]; s++) lcd.write(byte(5));
    radarBeep();
    delay(1200);
  }
  buzzersOff();
}
// ---------- IMPACT & SUCCESS ----------
void stageImpact() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("TARGET IMPACT ");
  lcd.write(byte(3));
  lcd.setCursor(0,1);
  lcd.print("AMBUSHED");
  continuousBeep(700);
  delay(200);
  continuousBeep(700);
}
void stageMissionSuccess() {
  missionCount++;
  saveMissionCount();
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("MISSION SUCCESS");
  lcd.setCursor(0,1);
  lcd.print("TOTAL RUNS: ");
  lcd.print(missionCount);
  successPulseGreen();
  for (int i = 0; i < 4; i++) {
    shortBeep();
    delay(250);
  }
  buzzersOff();
}
// ---------- RUN ONE MISSION ----------
void runMission() {
  aborted = false;
  stageInitialBoot();
  stageCommsLink();
  stageSystemReady();
  stageNavAlign();
  stageArmPrep();
  stageSafetyCheck();
  if (!stageCountdown()) return;
  stageLaunch();
  stageMidAirFlight();
  stageTargetSearch();
```

```
  stageTargetLocking();
  stageTargetLocked();
  stageMissileTrajectoryAnimation();
  stageImpact();
  stageMissionSuccess();
}
// ---------- POST-MISSION MENU ----------
void postMissionMenu() {
  int mode = 0; // 0 = Restart, 1 = Loop
  while (true) {
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("MODE: ");
    if (mode == 0) lcd.print("RESTART");
    else           lcd.print("LOOP");
    lcd.setCursor(0,1);
    lcd.print("3X=CHANGE 2X=OK");
    int presses = getPressCount(2500);
    if (presses == 3) {
      mode = !mode;
      shortBeep();
    } else if (presses == 2) {
      doubleBeep();
      break;
    } else {
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("WAITING FOR");
      lcd.setCursor(0,1);
      lcd.print("USER SELECTION");
      delay(800);
    }
  }
  if (mode == 0) {
    allLedsOff();
    analogWrite(LED_G1, 80);
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("RESTART MODE");
    lcd.setCursor(0,1);
    lcd.print("PRESS START");
    delay(2000);
    return;
  }
  // LOOP MODE
  stopLoop = false;
  while (!stopLoop) {
    runMission();
    for (int t = 10; t >= 1; t--) {
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("LOOP MODE ACTIVE");
      lcd.setCursor(0,1);
      lcd.print("STOP IN 10s: ");
      lcd.print(t);
      shortBeep();
      // Indicate loop mode using LED_G3
```

```
      analogWrite(LED_G3, 200);
      unsigned long start = millis();
      while (millis() - start < 400) {
        if (digitalRead(START_BUTTON) == LOW) {
          stopLoop = true;
          break;
        }
      }
      analogWrite(LED_G3, 0);
      unsigned long start2 = millis();
      while (!stopLoop && millis() - start2 < 500) {
        if (digitalRead(START_BUTTON) == LOW) {
          stopLoop = true;
          break;
        }
      }
      if (stopLoop) break;
    }
  }
  allLedsOff();
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("LOOP HALTED");
  lcd.setCursor(0,1);
  lcd.print("RETURNING...");
  delay(1600);
}
// ---------- SETUP & LOOP ----------
void setup() {
  lcd.init();
  lcd.backlight();
  pinMode(BUZZER1, OUTPUT);
  pinMode(BUZZER2, OUTPUT);
  pinMode(RELAY, OUTPUT);
  pinMode(START_BUTTON, INPUT_PULLUP);
  pinMode(LED_G1, OUTPUT);
  pinMode(LED_G2, OUTPUT);
  pinMode(LED_G3, OUTPUT);
  digitalWrite(RELAY, LOW);
  buzzersOff();
  allLedsOff();
  lcd.createChar(0, missileChar);
  lcd.createChar(1, radarChar);
  lcd.createChar(2, targetChar);
  lcd.createChar(3, explosionChar);
  lcd.createChar(4, successChar);
  lcd.createChar(5, blockChar);
  loadMissionCount();
}
void loop() {
  standbyMode();
  enterPassword();
  if (!authenticated) return;
  healthCheck();
  runMission();
```

```
  postMissionMenu();
}
```

4.2 Code installation:-

The successful operation of the Launch Control System depends on correct installation of software tools, proper library setup, correct wiring, and uploading the final program (Arduino sketch) into the Arduino Uno microcontroller. Below is the complete step-by-step procedure for code installation and uploading.

◈ Step 1: Install Arduino IDE

[1] Open browser and go to:

[2] Download Arduino IDE software (Windows / Mac / Linux).

[3] Install it on your computer.

[4] Open the Arduino IDE after installation.

◈ Step 2: Connect Arduino Uno to PC

[1] Use the USB Type-A to Type-B cable (Arduino Printer cable).

[2] Plug the Type-B side into the Arduino Uno.

[3] Plug the Type-A side into your laptop or PC.

[4] The Arduino will get power (LED on board will glow).

◈ Step 3: Select Board and Port

In Arduino IDE:

◈ Go to Tools → Board → Arduino AVR Boards → Arduino Uno

◈ Then select Tools → Port → COM x (where x is the detected Arduino port)

If Port is missing:

– Install CH340 or Arduino USB driver (auto-installed in latest IDE)

◈ Step 4: Add Required Libraries

Your project uses OLED/LCD (I2C) display and EEPROM storage.

To install libraries:

◈ Go to Sketch → Include Library → Manage Libraries

◈ Step 5: Open or Paste the Final Launch Code

- Create a New Sketch, File → New
- Paste the entire Final Launch Control System code (with passwords, relay, LEDs, LCD, buzzers, loop mode, PWM, EEPROM etc.)
- Ensure there are no missing braces or syntax errors.

Step 6: Verify the Code

- Click Verify / Compile in Arduino IDE
- IDE will check for errors and library linking
- If successful: "Done compiling" will appear

If errors appear:

✓ Check missing semicolons ;

✓ Ensure LiquidCrystal_I2C is installed

✓ Check relay/buzzer pin numbers in code

Step 7: Upload Code to Arduino

- Click Upload
- IDE will flash the microcontroller
- Wait until message appears:

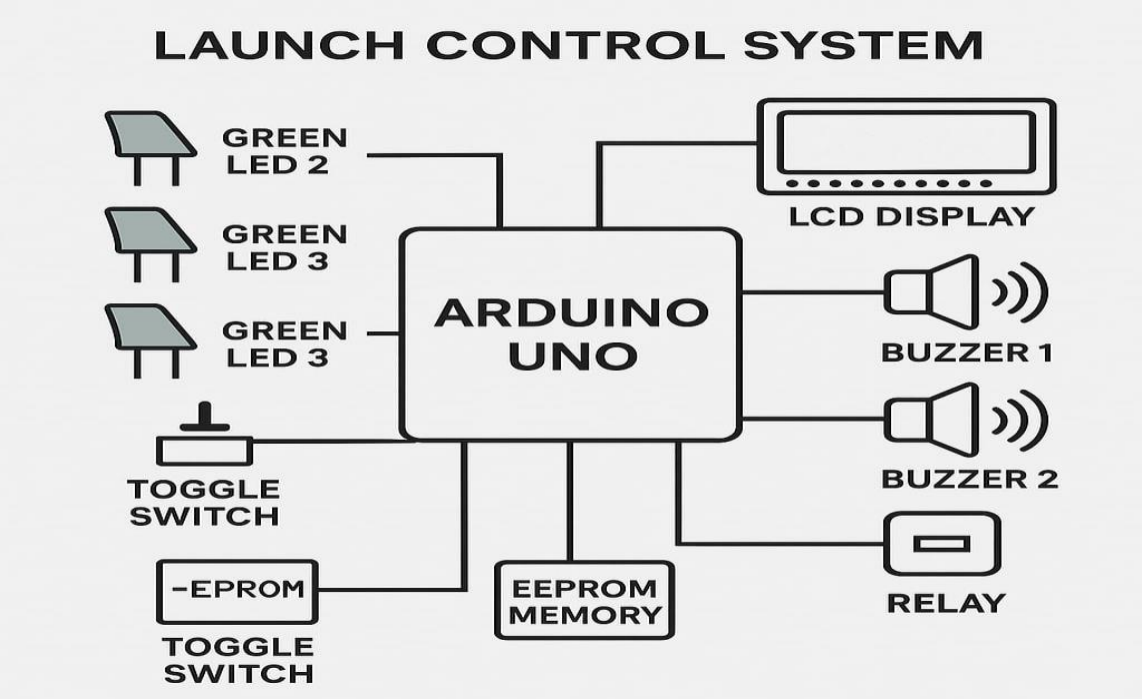
Step 8: First Run — Testing the System

After uploading:

- LCD should display:
- "SYSTEM STANDBY"
- LED_G1 will glow dimly
- Press toggle switch to enter password
- Enter password 1234 using multi-press logic
- System continues through each stage

Chapter 5

5.1 Block diagram of Launch Control System:



5.2 Description of block diagram:-

The Launch Control System consists of: - [1] Power Supply (Laptop/PC via USB) [2] Input Unit – Toggle Switch (Command Trigger & Password Input) [3] Processing Unit – Arduino Uno Microcontroller [4] Output Units – LCD, Relay, LEDs, Buzzers [5] Memory Unit – EEPROM for mission storing

1. Power Supply (Laptop / PC USB) - In this project, no external power adapter or battery is used. The Arduino receives power directly from Laptop/PC via USB cable, and Arduino supplies power to all connected components.

Arduino Uno	Powered via USB from Laptop/PC
LCD, LEDs, Buzzers, Relay	Powered from 5V pin of Arduino
Toggle Switch	Powered via Internal Pull-Up in Arduino
EEPROM	Internal to Arduino (no external power)

2.Arduino USB Power Specifications:- Voltage: 5V DC,Current: up to 500mA (enough for relay, LCD, LEDs, buzzers), Input Unit — Toggle Switch (User Control).The only input device used in this project is a single toggle switch, connected to Pin D2 using INPUT_PULLUP mode.

The same switch is used for:

Password entry	Press 1–4 times to enter digits
Abort launch	Press during countdown
Restart	Single press after mission
Loop Mode	Multiple presses for menu selection

3. Arduino Uno — Processing & Control Unit- Arduino acts as the central command system (similar to real missile Command Launch Unit - CLU).

Authentication	Verifies password
Sequence Control	Launch events in correct order
Safety Interlock	Abort and Safe Mode
Relay Control	Arm and ignition signal
PWM Output	LED blinking and buzzer tones
Display Output	Show status on LCD
EEPROM Storage	Mission success counter

4. Output Units Details - LCD Display (Human Machine Interface – HMI)-Shows all launch statuses on screen.Displays animations, countdown timer, target tracking, distance decreasing, and mission success.

Buzzers (Audio Alert System)

Soft Beep	Key press, authentication
Continuous Beep	Launch ignition
Repeated beep	Countdown warning
Double beep	Target locked
Success tones	Mission completed

Relay Module (Arming & Ignition Simulation)- Controlled by Arduino Pin D5 (HIGH/LOW).Pulses ON/OFF during ARM PREP stages.Remains HIGH during LAUNCH ACTIVE.Simulates real ignition and firing control signal.

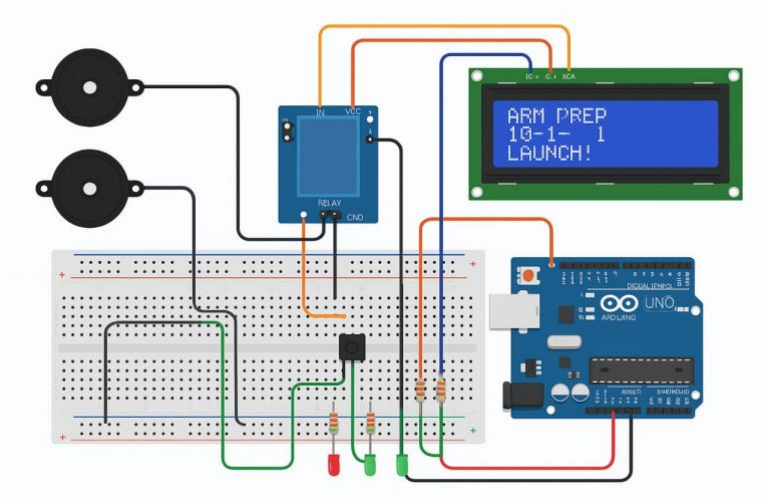
LED Indicators (Visual Feedback)

LED_G1	D3	Standby, System Ready, Navigation
LED_G2	D6	Arming, Countdown, Launch Active
LED_G3	D9	Mission Success, Loop Mode Indicator

5. EEPROM (Memory Storage Module)- EEPROM is built into Arduino. Stores mission count even when system is unplugged. Displays total runs at next startup. Mimics black box / launch logs used in real weapon systems.

Power from Laptop USB
↓
System Standby (LCD, LED_G1)
↓
Password Authentication (Toggle Switch)
↓
Health Check (Relay, LEDs, Buzzers)
↓
Arming (Relay Pulses, LED_G2, Buzzer)
↓
Countdown with Abort Option
↓
Launch Active (Continuous Relay and Buzzer)
↓
Tracking → Target Locked → Impact
↓
Mission Success → EEPROM Updated
↓
Restart / Loop Mode

5.3 Circuit diagram :-



5.4 Description of circuit diagram:-

The circuit diagram represents how all electronic components are connected to the Arduino Uno to simulate a real-time Launch Control System. This circuit uses USB power from a laptop, and the Arduino further powers the LCD, relay, LEDs, and buzzers. It is a low-power digital control circuit, suitable for demonstration and simulation.

Power Distribution in Circuit- The system is powered entirely via USB (5V DC) from Laptop/PC. Arduino's 5V pin provides regulated voltage to: ✓ LCD I2C Display ✓ Relay Module (5V Trigger) ✓ LEDs and Buzzers (through resistors).Arduino's GND pins are connected to the: ✓ Breadboard ground rail (common ground) ✓ Buzzer GND, Relay GND, LED GND ✓ LCD GND and Toggle Switch GND

Buzzer1	A3	Digital PWM sound generation
Buzzer2	D8 or D10	PWM-based audio alerts
Relay Module	D5	Arming and launch ignition simulation
Toggle Switch	D2	User input, password, abort, restart
LED_G1	D3	Standby / Navigation Ready
LED_G2	D6	Arming/Countdown/Launch
LED_G3	D9	Mission Success / Loop Mode
LCD SDA	A4	I2C Data Line
LCD SCL	A5	I2C Clock Line
LCD VCC	5V	Power supply
LCD GND	GND	Ground

LCD Display Wiring (I2C Communication)

LCD Pin	Arduino Pin
SDA	A4
SCL	A5
VCC	5V
GND	GND

Relay Module Wiring-

Relay Pin	Arduino Pin	Function
VCC	5V	Power
GND	GND	Common ground
IN	D5	Digital control signal (HIGH/LOW)
NO/NC	Not connected*	Simulation, not real load

Buzzer Connections- Buzzer 1 → A3 and GND(Used for countdown, missile tracking, successful lock)
Buzzer 2 → D8 (or D10) and GND(Used for launch alerts, continuous ignition tone)

Standby / Ready	D3 (LED_G1)	Dim glow / slow pulse
Launch Active	D6 (LED_G2)	Fast blink / steady ON
Mission Success	D9 (LED_G3)	Smooth fading PWM

Toggle Switch Wiring (Single Input Button)- Toggle switch one end → D2,Other end → GND,Arduino internally uses INPUT_PULLUP mode

Switch State	Logic Level
Not Pressed	HIGH
Pressed	LOW

Circuit Working Highlights- ✓ Arduino acts as central controller ✓ Toggle switch used for password input, abort, and restart ✓ Relay simulates ignition commands ✓ LCD provides full HMI-based interface ✓ Buzzers and LEDs act as warning and status indicators ✓ EEPROM stores number of successful missions ✓ All components share COMMON GND for stable operation

CHAPTER 6

6.1 Advantages:-

1. Demonstrates Real Missile Launch Logic- This project successfully simulates the actual stages in missile command and control.

2. Smart Use of Single Input for Multiple Functions- In real missile systems, fewer physical controls are used to avoid accidental activation. This project uses only one toggle switch to Enter password

3. Real-Time Human Machine Interface (HMI) Simulation- The 16x2 I2C LCD acts like a real Launch Control Panel, displaying.

4. Demonstrates Safety and Abort Mechanism- Just like in real missile operations, this system includes: Password security, Abort launch during countdown, Safe mode for unauthorized access

5. Realistic Relay Ignition Simulation The relay acts like missile ignition circuit, switching during Arming Preparation, Ignition & Launch, Deactivation after flight. This helps understand how relay circuits are used in missile ignition, automotive, PLC, and industrial applications.

6. Low-Cost Yet Highly Effective Project

It requires no high-voltage, no batteries, no programming shield, yet simulates realistic missile launch logic.

6.2 Applications:

Launch Control System project is a powerful demonstration of how electronics, automation, instrumentation, and safety control technologies are used in real-world defense, industrial, and engineering applications. It not only simulates missile launch sequencing but also showcases how microcontroller-based systems can be used to monitor, control, and automate critical operations.

1. **Missile Command and Control Simulation: -**This project can be used as a training simulator to demonstrate:

   - Secure access and authorization
   - Safety arming and ignition sequencing
   - Countdown and launch control
   - Target lock, trajectory tracking, and mission impact

   This makes it ideal for defense training institutes, engineering labs, and educational exhibitions.

2. Safety Arming and Interlocking Mechanism:-

The system demonstrates how safety protocols must be verified before activating dangerous machinery. The same logic is used in industries to prevent accidental activation of machines, ensuring human safety.

3. Automation and Industrial Control Systems:-

Relay-based control, PWM actuation, countdown automation, and digital signal control in this project represent real applications in:

✓ PLC-based Industrial Process Control

✓ Conveyor Belt & Robotic Arm Automation

✓ Delay & Timer-based Heater/Furnace Control

✓ Power Station Safety Control

✓ Emergency Stop and Lockout Mechanisms

It demonstrates sequential automation and machine safety logic, widely used in factories.

4. Human Machine Interface (HMI) Demonstration:-

The LCD in this project simulates an Operator Control Panel, similar to: Control Room Displays

- Missile Launch Consoles
- Monitoring Dashboards
- Industrial HMI Panels

5. Signal Processing and Instrumentation Concepts:-The project demonstrates:

✓ PWM control (LED fading, buzzer tones)

✓ Relay switching (signal conditioning)

✓ Sensor/Signal feedback via toggle switch

✓ EEPROM-based data logging

7.1 Future Expansion:-

Although the current model successfully simulates a microcontroller-based Launch Control System, there are several exciting ways this project can be enhanced to make it even more realistic, technologically advanced, and industry-relevant. These improvements can help transform this basic prototype into a smart, IoT-based, AI-enabled missile command system, closely resembling real-world aerospace and defense applications.

1. Wireless Control and Remote Monitoring:-

Improvement Description

Wi-Fi Control using ESP32- Trigger launch from mobile or remote location

GSM/4G Module- Receive launch authorization via SMS or cloud

Bluetooth App Control- Operate, reset, or abort using Android app

IoT Dashboard- Monitor countdown, target lock, and success over the internet

This converts the project into a smart, remote-controlled launch platform.

2. GPS-Based Real-Time Target Tracking:-By integrating GPS and ultrasonic/IR sensors, the system could:

✓ Track actual missile movement

✓ Display real-time changing distance on LCD

✓ Show latitude-longitude position

✓ Provide trajectory correction feedback

3. AI-Based Target Recognition :-Using AI, camera modules, and ML algorithms:

✓ Detect target using image processing

✓ Automatically lock target

✓ Display target image on OLED/Nextion HMI

✓ Trigger launch only when target is verified

4. Touch-Based Control Panel:-Instead of a 16x2 LCD, higher-level displays can be used:

Display Type Feature

TFT / OLED- Color graphics and missile animations

Nextion Display- Touchscreen buttons for launch, abort, replay

GUI-Based PC Dashboard- Full simulation panel with indicators and logs

5. Multi-Stage Launch and Multi-Missile Simulation:-

Future expansion can include: ✓ Multiple missile launch pads

✓ Parallel launch countdowns

✓ Stage-1 and Stage-2 rocket separation

✓ Live local radar feed simulation

✓ Intercept and defense simulation

6. Mission Log Export & Data Recording:-By adding SD card or cloud logging:

✓ Record number of successful and failed missions

✓ Export data to Excel (.csv)

✓ Generate mission summary reports

✓ Track date and time for each launch (RTC module)

7. Industrial Applications:-This system can be modified for:

✓ Industrial automation startup sequence

✓ Emergency shutdown of machines

✓ Disabling unauthorized machine usage

✓ Controlled ignition systems for labs and chemical reactors

✓ PLC and SCADA-based safety interlocking

CHAPTER 8

8.1 Conclusion:-

The Launch Control System using Arduino successfully demonstrates the fundamental concept of missile launch sequencing, safety arming mechanisms, authentication, countdown control, trajectory monitoring, and mission outcome simulation. It integrates key concepts from Electronics, Instrumentation, Embedded Systems, Control Engineering, and Defense Simulation Technology, providing a realistic functional model of how real missile launch systems operate.Throughout this project, we implemented several essential engineering principles including digital authentication (password entry), relay-based ignition simulation, PWM-based visual and audio alerts, LCD-based Human Machine Interface (HMI), and EEPROM-based mission data storage. The system also includes advanced features such as abort functionality, loop mode execution, missile tracking animation, distance-based target engagement, and post-launch mission analysis — closely mimicking real military launch procedures.This project demonstrates sequential automation, where events are executed in a controlled order:

System Standby → Authentication → Self Test → Navigation Alignment → Arming Sequence → Countdown → Launch → Flight Simulation → Target Tracking → Impact → Mission Success → Restart/Loop Mode

By using Arduino as the central controller, the system efficiently manages input processing, stage transitions, user interaction, hardware activation, and decision-making. The use of single toggle switch multi-function input emphasizes input optimization, reducing hardware complexity while improving system intelligence — just like in real missile command systems where minimal physical controls are used for security.The system fulfills all its objectives — including secured access, controlled launch sequence, automated tracking, user feedback, and mission result logging. It is cost-effective, easy to build, and ideal for Tech Fest demonstrations, academic projects, and training simulations.In conclusion, this project is a combination of practical engineering, logical programming, and real-world system simulation, making it an excellent representation of how electronics and instrumentation engineering concepts can be applied to modern defense and automation technologies.